



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 08 - DE CHATBOTS A AGENTES
AUTÔNOMOS



OBJETIVOS DA AULA

- ❑ Analisar a transição arquitetural de um sistema de fluxo de dados para um de fluxo de controle (Agente).
- ❑ Discutir o papel do LLM como um motor de raciocínio plugável, capaz de traduzir intenção em chamadas de função estruturadas (Tool Calling).
- ❑ Aprofundar no padrão de ciclo de vida de um agente, o ReAct (Reason + Act), e seu processo iterativo de observação, pensamento e ação.
- ❑ Discutir as implicações do paradigma de agentes para o desenvolvedor Java: design de Tools como APIs, gerenciamento de estado e tratamento de erros.
- ❑ Mapear os conceitos teóricos de um agente para os componentes específicos do Langchain4j que serão implementados.



DE FLUXO DE DADOS A FLUXO DE CONTROLE

DE FLUXO DE DADOS A FLUXO DE CONTROLE

- ❑ Até então: RAG como um Padrão de Fluxo de Dados
 - ❑ Nosso sistema RAG é, em sua essência, um pipeline de transformação de dados. O fluxo é linear, previsível e unidirecional:
 - ❑ A pergunta do usuário (Query) é o ponto de partida.
 - ❑ O RetrievalAugmentor enriquece essa Query com dados externos (Context).
 - ❑ O LLM recebe o Context + Query e sua única função é sintetizar essa massa de texto em uma resposta (Answer).
 - ❑ Neste modelo, o LLM é um processador de texto sofisticado
 - ❑ O controle do fluxo da aplicação está inteiramente no nosso código Java.
 - ❑ Nós decidimos quando chamar o RetrievalAugmentor e quando chamar o LLM.

AGENTE COMO UM PADRÃO DE FLUXO DE CONTROLE

- ❑ Um agente inverte essa relação. O fluxo deixa de ser linear e passa a ser cíclico e dinâmico, orquestrado pelo LLM:
 - ❑ A solicitação do usuário (Intent) é o gatilho.
 - ❑ O LLM analisa a intenção e, em vez de gerar uma resposta final, ele pode gerar uma diretiva de execução (uma chamada de Tool).
 - ❑ Nosso código Java atua como um executor de capacidades, invocando a Tool solicitada.
 - ❑ O resultado da execução é retornado ao LLM, que o utiliza como uma nova observação para decidir o próximo passo.
- ❑ Neste novo modelo, o LLM se torna o cérebro que controla o fluxo da aplicação, e nosso código Java se torna um conjunto de capacidades de negócio que o LLM pode invocar sob demanda.
 - ❑ A lógica de orquestração é delegada ao modelo de linguagem.

O MECANISMO CENTRAL: TOOL CALLING

O MECANISMO CENTRAL: TOOL CALLING

- ❑ A capacidade que permite a um LLM controlar o fluxo é chamada de Tool Calling. Modelos modernos são treinados não apenas para gerar texto, mas também para gerar, quando apropriado, um objeto JSON estruturado que corresponde a uma especificação de função.
- ❑ O Contrato de Interface:
 - ❑ No início da interação, nossa aplicação fornece ao LLM um "manifesto" das Tools disponíveis. Para cada Tool, descrevemos seu propósito, os parâmetros que ela aceita (nome, tipo, se é obrigatório, etc) e o que ela retorna.
 - ❑ Quando o usuário faz uma solicitação, o LLM compara a intenção semântica do usuário com as descrições das Tools em seu manifesto.
 - ❑ Se houver uma correspondência, a saída do LLM não é uma frase, mas um JSON que representa uma chamada de função.

EXEMPLO DE INTERAÇÃO COM TOOL CALLING

- ❑ Cenário
 - ❑ O usuário diz: "Preciso cancelar minha reserva 12345, meu sobrenome é Silva."
- ❑ Saída do LLM (enviada para a aplicação)
 - ❑ Nossa aplicação Java recebe o JSON abaixo, o desserializa e executa o método Java correspondente. O LLM não executa código; ele delega a execução de forma estruturada.

```
{  
  "tool_name": "cancelarReserva",  
  "arguments": {  
    "idReserva": 12345,  
    "sobrenome": "Silva"  
  }  
}
```


O CICLO REACT (REASON + ACT)

O CICLO REACT (REASON + ACT)

- ❑ O comportamento de um agente é governado por um loop de raciocínio. O padrão ReAct formaliza este processo, permitindo que o agente decomponha problemas complexos em passos intermediários de pensamento e ação.
- ❑ Cada iteração do ciclo é uma chamada ao LLM que produz um "pensamento" e uma "ação".

Passo	Descrição Técnica	Exemplo
Observação	O estado atual do sistema. Na primeira iteração, é a solicitação do usuário. Nas iterações subsequentes, é o resultado da execução da Tool anterior.	User Input: "Quero um voo barato para Paris no próximo mês."
Pensamento (Reason)	O LLM gera uma "cadeia de pensamento" (Chain of Thought). Ele analisa a observação, avalia o objetivo final e planeja a próxima ação. Este é o passo de raciocínio.	Thought: "O usuário quer um voo para Paris. 'Barato' e 'próximo mês' são ambíguos. Preciso pedir esclarecimentos sobre o orçamento e as datas exatas."
Ação (Act)	O LLM externaliza sua decisão. A ação pode ser: 1. Chamar uma Tool: Gerar o JSON para executar uma função. 2. Responder ao Usuário: Gerar uma pergunta de esclarecimento ou a resposta final.	Action: Respond("Qual é o seu orçamento máximo e em quais datas de 'próximo mês' você gostaria de viajar?")

- ❑ Este ciclo se repete até que o LLM, em sua fase de Pensamento, determine que o objetivo original do usuário foi alcançado e a Ação final seja gerar uma resposta conclusiva.

UMA TAREFA “REACT” EM VÁRIOS CICLOS

UMA TAREFA “REACT” EM VÁRIOS CICLOS

- ❑ Cenário
 - ❑ "Verifique a disponibilidade do pacote Aventura Amazônia e, se houver vagas, adicione a opção de 'seguro de viagem'."
- ❑ Ciclo 1:
 - ❑ Observação:
 - ❑ User: "Verifique a disponibilidade do pacote Aventura Amazônia e, se houver vagas, adicione a opção de 'seguro de viagem'."
 - ❑ Pensamento:
 - ❑ O usuário tem duas intenções: verificar disponibilidade e depois adicionar um seguro. A segunda depende da primeira. A primeira ação é verificar a disponibilidade. A Tool 'verificarDisponibilidade(nomePacote)' parece adequada.
 - ❑ Ação:
 - ❑ Tool Call: verificarDisponibilidade(nomePacote="Aventura Amazônia")

UMA TAREFA “REACT” EM VÁRIOS CICLOS

- ❑ Ciclo 2:
 - ❑ Observação:
 - ❑ Tool Output: "{ 'disponivel': true, 'vagas': 15, 'idPacote': 'AMZ-001' }"
 - ❑ Pensamento:
 - ❑ A verificação de disponibilidade foi bem-sucedida; há 15 vagas. Agora preciso executar a segunda parte da solicitação: adicionar o seguro. A Tool 'adicionarSeguro(idPacote)' é a correta. O ID do pacote foi retornado pela ferramenta anterior.
 - ❑ Ação:
 - ❑ Tool Call: adicionarSeguro(idPacote="AMZ-001")

UMA TAREFA “REACT” EM VÁRIOS CICLOS

- ❑ Ciclo 3:
 - ❑ Observação:
 - ❑ Tool Output: "{ 'status': 'sucesso', 'seguroAdicionado': true, 'precoFinal': 4850.00 }"
 - ❑ Pensamento:
 - ❑ Ambas as ações foram concluídas com sucesso. A tarefa está finalizada. Agora preciso informar o usuário sobre o resultado de forma clara.
 - ❑ Ação:
 - ❑ Final Answer: "Ótima notícia! O pacote Aventura Amazônia tem disponibilidade. Adicionei a opção de seguro de viagem, e o preço final ficou em R\$ 4.850,00. Posso confirmar a reserva para você?"

IMPLICAÇÕES PARA O DESENVOLVEDOR JAVA



IMPLICAÇÕES PARA O DESENVOLVEDOR JAVA

- ❑ O Contrato de Tool: A API do seu Negócio para a IA
 - ❑ Um método anotado com `@Tool` não é apenas um método Java; ele se torna um endpoint de uma API que será consumida por um cliente não-humano (o LLM).
 - ❑ A descrição na anotação (`@Tool("descrição...")`) é a documentação da sua API. Ela deve ser clara, precisa e inequívoca. O LLM depende exclusivamente dessa descrição para decidir se e como usar a Tool. Uma descrição ruim levará a chamadas incorretas ou à não utilização da Tool.
- ❑ Boa Prática:
 - ❑ Pense em Tools como endpoints de uma API REST. Elas devem ser bem definidas, ter um propósito único e claro, e seus parâmetros devem ser tipos simples e bem descritos.
 - ❑ Aplica práticas de prompt engineering ao descrever a sua Tool.

IMPLICAÇÕES PARA O DESENVOLVEDOR JAVA

- ❑ O Papel Crítico da Memória
 - ❑ Em um sistema de agentes, a ChatMemory evolui de um simples histórico de diálogo para um log de transações de estado.
 - ❑ Ela não armazena apenas o que foi dito, mas também o que foi feito. O resultado de uma Tool (seja um sucesso ou um erro) é adicionado à memória e se torna parte do contexto para a próxima iteração do ciclo ReAct.
 - ❑ Isso é o que permite ao agente realizar tarefas em várias etapas. O resultado da verificarDisponibilidade (incluindo o idPacote) foi armazenado na memória, permitindo que o LLM o utilizasse no passo seguinte para chamar a adicionarSeguro. Sem a memória, o agente sofreria de "amnésia" entre as ações.

IMPLICAÇÕES PARA O DESENVOLVEDOR JAVA

- ❑ Erros em Tools não são Exceções, são Observações
 - ❑ Em uma aplicação tradicional, uma `BookingNotFoundException` lançada por um serviço resultaria em uma resposta de erro 500 para o cliente.
 - ❑ Em um sistema de agentes, isso é um erro de design. A exceção deve ser capturada pela nossa camada de execução de Tools.
 - ❑ O erro deve ser transformado em uma mensagem descritiva (ex: "Erro: A reserva com o ID 12345 não foi encontrada.") e retornado ao LLM como a próxima Observação no ciclo ReAct.
 - ❑ O LLM pode então raciocinar sobre o erro. Seu próximo Pensamento pode ser: "A ferramenta falhou porque a reserva não existe. A ação correta agora é informar o usuário sobre o erro e perguntar se ele gostaria de tentar um número de reserva diferente."
 - ❑ Isso transforma o tratamento de exceções de um mecanismo de finalização de fluxo para uma parte integrante do fluxo de controle conversacional.

MAPEAMENTO PARA O LANGCHAIN4J

MAPEAMENTO PARA O LANGCHAIN4J

- ❑ A teoria dos agentes se materializa em componentes concretos no Langchain4j, que se integram perfeitamente com o ecossistema Quarkus.

Conceito do Agente	Componente Langchain4j / Quarkus	Papel Técnico
Manifesto de Tools	Uma classe CDI (ex: <code>@ApplicationScoped</code>) com métodos anotados com <code>@Tool</code> .	Cada método anotado é uma capacidade que o agente pode executar. A string de descrição na anotação <code>@Tool("...")</code> é o que o LLM usa para entender o que o método faz.
Orquestrador do Ciclo ReAct	A interface <code>AiService</code> anotada com <code>@RegisterAiService(tools = MinhaClasseDeTools.class)</code> .	O Langchain4j, por baixo dos panos, gerencia o loop ReAct. Ele envia a lista de Tools para o LLM, interpreta a resposta (seja texto ou uma chamada de Tool), executa a Tool se necessário, e realimenta o resultado no LLM.
Estado da Conversa	Uma implementação de <code>ChatMemory</code> (ex: <code>MessageWindowChatMemory</code>).	Armazena o histórico de interações (usuário, agente, resultados de Tools). É essencial para que o Pensamento do LLM tenha contexto sobre os ciclos anteriores.
Motor de Raciocínio	Uma implementação de <code>ChatLanguageModel</code> (ex: <code>OllamaChatModel</code>) com suporte a Tool Calling.	O cérebro da operação. É o componente que efetivamente gera os "Pensamentos" e as "Ações" (em formato JSON) com base nas observações.

RESUMO E PRÓXIMOS PASSOS

RESUMO E PRÓXIMOS PASSOS

- ❑ O que Aprendemos Hoje?
 - ❑ A transição de um sistema de fluxo de dados para um Agente (fluxo de controle) representa uma mudança fundamental na arquitetura, onde o LLM passa a orquestrar a lógica de negócio.
 - ❑ O mecanismo de Tool Calling é a base que permite ao LLM traduzir a intenção do usuário em chamadas de método estruturadas e executáveis.
 - ❑ O ciclo ReAct é o padrão de execução iterativo que permite a um agente decompor problemas, usar Tools e resolver tarefas complexas em múltiplos passos.
 - ❑ Projetar agentes exige pensar em Tools como APIs, gerenciar o estado da conversa como um log de transações e tratar erros como parte do fluxo de controle.
- ❑ Próxima Aula:
 - ❑ Vamos traduzir essa teoria em código. Implementaremos nossa primeira classe de Tools, expondo métodos Java de negócio para o nosso agente. Vamos definir Tools para consultar e manipular dados de reservas, transformando nosso AiService de um simples respondente para um agente funcional.

ATÉ A PRÓXIMA!